

Maritime CargoService: Sprint 2

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 23, 2026

1. Introduction

Il punto di partenza di questo Sprint è il prototipo realizzato nello Sprint 1, documentato al seguente [link](#)^o.

Lo Sprint 1 ha prodotto un prototipo eseguibile del **cargoservice** in cui il ciclo principale di carico viene gestito tramite attori QAK, con componenti simulati per IOPort, sonar e LED, e con il **cargorobot** modellato come adapter del servizio esterno **robotsmart26**.

L'architettura finale dello Sprint 1, riportata di seguito, costituisce il riferimento di partenza per lo Sprint 2.

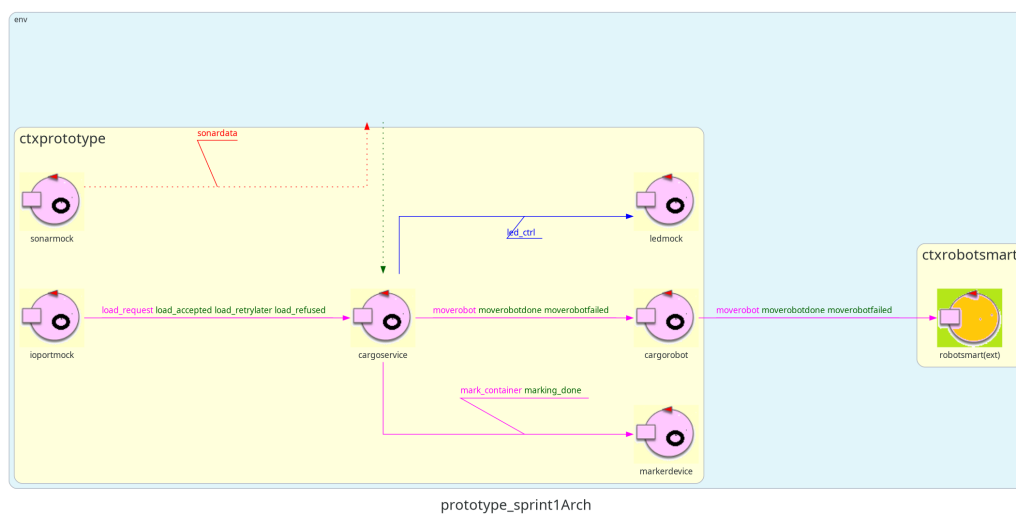


Figure 1: Architettura finale definita nello Sprint 1.

1.1. Goal dello Sprint 2

Il goal dello Sprint 2 è quello definito nella pagina di sintesi dello Sprint 1, ovvero:

evolvere il prototipo sviluppato sostituendo progressivamente i componenti simulati con le rispettive implementazioni reali:

- IOPort come Web GUI;
- sonar e LED collegati al PicoW;
- completare la gestione dello stato **Out of service** integrando il sonar reale, implementando la verifica della misura e l'interruzione/ripresa del movimento del cargorobot.

Si stima una durata di circa **30 ore** complessive di lavoro, corrispondenti a circa 10 ore per ciascun componente del gruppo.

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o.

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della hold riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della hold.
- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{\text{FREE}}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold;
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente;
 - il messaggio "**Out of service**" se il sensore sonar misura la distanza del container dal sonar stesso $D > D_{\text{FREE}}$ per almeno 3 secondi.

3. Requirement analysis

L'analisi dei requisiti è stata svolta nello [Sprint 0°](#) e approfondita nello [Sprint 1°](#). Tali risultati vengono assunti come validi anche per lo Sprint 2.

4. Problem analysis

Come definito precedentemente, bisogna sostituire progressivamente i componenti simulati con componenti accessibili attraverso tecnologie compatibili con la loro natura.

Il nostro team di sviluppo ha utilizzato una scheda **ESP32** invece del **Raspberry Pi Pico W** indicato dalla committente. L'architettura resta sostanzialmente invariata: anche ESP32 ha connettività ad internet, supporta MicroPython, ha un LED, dispone dei GPIO necessari per il sensore HC-SR04 e offre prestazioni generalmente superiori.

L'unica differenza riguarda aspetti implementativi, come la diversa numerazione dei pin e l'assenza di moduli specifici del Pico W (ad esempio rp2). Tuttavia, tali differenze non incidono sul progetto.

4.1. Osservabilità dello stato della Hold e considerazioni sulla Web GUI

Nello Sprint 1, l'IOPort era modellato come un attore QAK “mock” all'interno dello stesso contesto del cargoservice. Questa vicinanza tecnologica permetteva all'IOPort di reagire direttamente ai singoli eventi e messaggi asincroni scambiati sulla rete di attori.

Adesso l'IOPort evolve in una Web GUI eseguita in un browser web esterno. Questa transizione fisica e architetturale solleva una problematica fondamentale che rende i messaggi QAK nativi inadeguati per l'aggiornamento dell'interfaccia:

Il modello QAK descrive il comportamento del sistema attraverso lo scambio di messaggi, mentre una Web GUI deve mostrare lo stato corrente del sistema in un preciso momento. Ricostruire tale stato a partire dalla sequenza dei messaggi renderebbe la GUI complessa e dipendente dalla logica interna degli attori. Per questo viene introdotta una rappresentazione serializzata dello stato, aggiornata dal cargoservice e utilizzata dalla GUI solo per la visualizzazione. In questo modo, la gestione della logica rimane separata dalla presentazione grafica.

Lo stato dinamico del **sistema** viene perciò astratto e centralizzato in una rappresentazione serializzabile in formato **JSON** (mediante un metodo apposito), direttamente interpretabile dal motore del client da noi scelto, ovvero **JavaScript**.

```
public static String toJson(String serviceState, String workingState, boolean
ioPortOccupied, int reservedSlot) {
    return INSTANCE.doToJson(serviceState, workingState, ioPortOccupied,
reservedSlot);
}
```

Esempio di JSON creato dal metodo `toJson` :

```

{
  "serviceState": "engaged",          // engaged, disengaged
  "workingState": "Service working", // Service working, Out of service
  "ioPort0occupied": true,            // true o false
  "reservedSlot": 2,                 // slot riservato per l'operazione
  corrente
  "slots": {                          // occupied, reserved, free
    "slot1": "occupied",
    "slot2": "reserved",
    "slot3": "free",
    "slot4": "occupied"
  }
}

```

Il **cargoservice** si assume la responsabilità di rigenerare questa stringa JSON e di notificarla all'esterno ogni volta che si verifica una variazione significativa del sistema (es. transizione tra gli stati engaged/disengaged, variazione delle distanze del sonar, ingresso/uscita dallo stato Out of service o completamento del deposito).

Il codice completo dell'attore **cargoservice** è disponibile al seguente [link](#).

4.2. Protocolli per l'esposizione dello stato

Una volta stabilito che lo stato del **cargoservice** e della **Hold** debbano essere rappresentati come una risorsa osservabile, occorre individuare il protocollo più adatto per renderla disponibile dalla Web GUI.

Le principali alternative sono:

1. Utilizzare il protocollo **HTTP tradizionale**, interrogando periodicamente il **cargoservice** mediante richieste GET (polling). Tale soluzione risulta però inefficiente, poiché genera richieste anche quando lo stato del sistema rimane invariato, aumentando il traffico di rete e il carico sui componenti coinvolti.
2. Utilizzare il protocollo **MQTT** pubblicando un messaggio ogni volta che lo stato cambia. MQTT, nonostante permetta notifiche push, richiede di modellare lo stato come una sequenza di pubblicazioni su topic, il che risulterebbe meno naturale poiché lo stato da rappresentare è più una risorsa persistente.
3. Utilizzare il protocollo **CoAP**. Questo approccio risulta particolarmente adatto, poiché il runtime QAK rende gli attori risorse CoAP nativamente "osservabili". Observe è un'estensione del protocollo CoAP che permette una comunicazione asincrona (di tipo publish-subscribe) permettendo al "client" di venire notificato dagli attori (tramite la funzione nativa di QAK `updateResource`) ogni volta che lo stato cambia (senza quindi dover fare polling). Il **cargoservice** si limiterà ad aggiornare lo stato, il client si "iscriverà" ad esso ricevendo automaticamente la nuova rappresentazione solo quando necessario. Come definito in precedenza, lo stato sarà esposto sotto forma di documento JSON.

Esempio: transizione stato engaged/disengaged

```

State disengaged {
    println("cargoservice | DISENGAGED: waiting for requests...") color blue
    [# val statusJson = Hold.toJson(CargoState, if(ServiceWorking) "Service
working" else "Out of service", IOPortOccupied, ReservedSlotId) #]
    updateResource [# statusJson #]
}
// ...
State engaged {
    println("cargoservice | ENGAGED: evaluating next step...") color blue
    [# val statusJson = Hold.toJson(CargoState, if(ServiceWorking) "Service
working" else "Out of service", IOPortOccupied, ReservedSlotId) #]
    updateResource [# statusJson #]
}
Goto do_robot_job if [# IOPortOccupied && CargoState == "engaged" &&
ServiceWorking #] else wait_for_container

```

Rimane da risolvere il problema della comunicazione tra il mondo QAK e il browser.

Una prima possibilità sarebbe consentire alla Web GUI di osservare direttamente la risorsa CoAP del cargoservice.

Tale soluzione non risulta però praticabile poiché i browser moderni non supportano nativamente il protocollo CoAP.

Una seconda possibilità consiste nell'introdurre un componente intermedio che osservi la risorsa CoAP e renda disponibili gli aggiornamenti mediante protocolli compatibili con il browser, lasciando indipendenti sia il cargoservice sia la GUI.

Viene perciò introdotto un server chiamato **ioport-backend** con il compito di:

- osservare la risorsa CoAP del **cargoservice**, ricevendo gli aggiornamenti in formato JSON tramite il meccanismo **Observe**;
- esporre tali informazioni alla Web GUI tramite un protocollo la cui scelta è definita nella sezione successiva;
- inoltrare al **cargoservice** le richieste provenienti dalla GUI.

4.3. Realizzazione dell'IOPort come Web GUI

L'IOPort richiesto dalla committente è costituito logicamente da:

- un pushbutton, utilizzato dal cliente per inviare una **load_request**;
- un display, utilizzato per mostrare la risposta alla richiesta e lo stato corrente del servizio e della hold.

La sua implementazione viene suddivisa in due parti:

- una pagina web eseguita nel browser (disponibile al seguente [link](#)[◦]).
- un server intermedio che collega il browser al sistema QAK (disponibile al seguente [link](#)[◦]) e che funge da web server per la pagina web. Si è scelto di utilizzare Javalin, un framework leggero per la creazione di web server in Java, che permette di gestire

facilmente le richieste HTTP. Per implementare il meccanismo di Observe di CoAP si utilizzano specifiche funzioni di libreria.

4.3.1. Invio della `load_request`

Quando il cliente preme il pulsante per la richiesta di carico, la GUI invia una richiesta HTTP `POST` al server intermediario.

Il server traduce tale richiesta in una `load_request` indirizzata al **cargoservice** e attende una delle risposte già definite nello Sprint 0.

La risposta viene quindi restituita alla GUI e mostrata sul display.

Questa soluzione mantiene invariato il protocollo applicativo QAK già validato nello Sprint 1: HTTP viene utilizzato soltanto nel tratto browser-server, mentre l'interazione con il **cargoservice** continua a essere espressa mediante i messaggi del modello.

4.3.2. Aggiornamento del display

Per effettuare l'aggiornamento dello stato sul display si potrebbe:

1. utilizzare un polling periodico da parte della GUI, che interroga il backend per verificare eventuali variazioni dello stato ricavate tramite CoAP dal **cargoservice**, anche in questo caso, notoriamente pesante come meccanismo;
2. utilizzare **WebSocket**, un meccanismo che realizza una comunicazione bidirezionale permanente, permettendo al server di notificare immediatamente ogni aggiornamento ricevuto tramite CoAP. Si sceglie tale soluzione in quanto più efficiente e reattiva.

Ora il server, dopo aver ricevuto l'aggiornamento dello stato del **cargoservice** può inoltrare immediatamente le informazioni alla GUI senza che quest'ultima debba richiederle periodicamente.

4.3.3. Evoluzione dell'IOPort

Nei primi Sprint l'IOPort era stata modellata come un attore QAK per simulare il comportamento dell'interfaccia utente durante la prototipazione del sistema, aiutando il team a testare velocemente l'interazione con il cargoservice. Con l'introduzione della Web GUI questa modellazione non risulta più necessaria. Il precedente attore QAK viene perciò rimosso e sostituito dalla suddetta architettura client-server composta da frontend e backend (il quale interagisce a sua volta con il cargoservice).

Il backend assume il ruolo precedentemente svolto dall'attore IOPort, quindi traduce le richieste ora inviate della GUI nelle corrispondenti Request verso il cargoservice e propaga le risposte e gli aggiornamenti.

Il contesto "ctxioport" rappresentava il nodo di esecuzione dell'attore ioport ma, poichè ora l'interfaccia utente è realizzata come applicazione web esterna al sistema QAK (e quindi come nodo indipendente), essa non necessita più di un context dedicato.

4.4. Integrazione del sonar reale

Nello sprint precedente, il sonar era stato modellato tramite l'attore `sonarmock`, il quale simulava il comportamento del dispositivo fisico generando eventi di tipo `sonardata` intercettati dal `cargoservice`.

Con l'introduzione del sonar reale, collegato a un dispositivo ESP32, questo approccio non è più applicabile. Infatti non è possibile eseguire codice QAK direttamente da ESP32 e comunicare tramite messaggi verso il `cargoservice`. Verrà quindi realizzato uno script in un linguaggio da esso comprensibile (**MicroPython**) con il compito di acquisire periodicamente la distanza rilevata dal sensore e rendere disponibili tali informazioni al sistema distribuito.

È adesso necessario adottare un protocollo che garantisca leggerezza, semplicità di integrazione e interoperabilità tra componenti eterogenei. Per questi motivi viene scelto MQTT, un protocollo basato sul modello publish/subscribe, particolarmente adatto a dispositivi con risorse limitate e scenari IoT. MQTT richiede la presenza di un componente intermediario (definito **broker**, per cui si sceglie di utilizzare Mosquitto) che si occupi di ricevere i messaggi pubblicati dall'ESP32 e gestirne la distribuzione al `cargoservice`.

Le misurazioni rilevate dal sonar vengono quindi pubblicate dall'ESP32 su un topic MQTT dedicato (detto `sonardata`), al quale i componenti interessati del sistema possono iscriversi per ricevere gli aggiornamenti in modo disaccoppiato rispetto al dispositivo fisico.

Per far sì che il `cargoservice` riceva i messaggi, una soluzione possibile sarebbe quella di introdurre un componente dedicato `sonaradapter`, simile al `sonarmock` dello Sprint 1, che svolge il compito di integrazione tra il dispositivo fisico e il sistema esistente. Esso potrebbe ricevere le misurazioni pubblicate dall'ESP32 tramite MQTT e inoltrare le informazioni al `cargoservice` mediante `dispatch`, modellando quindi una comunicazione affine a quella dei mock. Tuttavia, considerando che **QAK supporta nativamente il protocollo MQTT** (consentendo a un **attore** di iscriversi o di emettere eventi su un topic), tale proposta creerebbe dell'overhead di comunicazione. Risulta infatti più opportuno (e più semplice) modificare il `cargoservice` affinché, al posto di ricevere messaggi da un qualche attore "sonar", stabilisca la connessione al broker tramite la dichiarazione `mqttBroker` e riceva direttamente i messaggi MQTT sul topic dedicato.

Il codice dell'ESP32 che gestisce il sonar si trova al seguente [link](#)^o.

4.5. Validazione temporale delle misure sonar

Da requisiti, una singola misura del sonar non comporta necessariamente una transizione di stato. Sia la **presenza del container** sia la condizione di **Out of service** devono essere riconosciute solo quando la relativa condizione permane per almeno **tre secondi** consecutivi.

Dal punto di vista applicativo è quindi necessario distinguere tre intervalli di misura:

1. $D < DFREE / 2$ che indica una possibile presenza del container davanti all'IOPort;
2. $D > DFREE$ che indica una possibile condizione di malfunzionamento del sistema di carico (Out of service);
3. valori intermedi, che non soddisfano nessuna delle due condizioni precedenti.

Si è scelto un valore di `DFREE` pari a 20 cm.

Una possibile soluzione sarebbe demandare questa validazione direttamente all'ESP32, facendogli pubblicare solamente eventi già validati temporalmente. In questo modo però l'accoppiamento del dispositivo con il dominio applicativo aumenterebbe notevolmente.

Si preferisce pertanto mantenere l'ESP32 come semplice componente di acquisizione dati. Sarà quindi il `cargoservice` a dover memorizzare:

1. l'istante in cui viene rilevata per la prima volta la condizione $D < DFREE/2$;
2. l'istante in cui viene rilevata per la prima volta la condizione $D > DFREE$;
3. l'annullamento del conteggio quando la misura torna nell'intervallo normale.

Solo quando una delle due condizioni permane per almeno tre secondi consecutivi viene effettivamente effettuata una transizione di stato.

```
// condizione PRESENZA CONTAINER verificata
if (Dist < DFreeDiv2) {
    if (ContainerPendingStart < 0L) {

        // inizio del conteggio dei 3 secondi
        ContainerPendingStart = Now

    } else if (Now - ContainerPendingStart >= 3000L) {

        // Condizione sostenuta per 3s -> Container rilevato
        IOPortOccupied = true
        ContainerPendingStart = -1L
    }
} else {
    ContainerPendingStart = -1L
    IOPortOccupied = false
}

// condizione OUT OF SERVICE verificata
if (Dist > DFree) {
    if (ServiceWorking) {

        if (OutOfServicePendingStart < 0L) {
            // inizio del conteggio dei 3 secondi
            OutOfServicePendingStart = Now

        } else if (Now - OutOfServicePendingStart >= 3000L) {

            // guasto sostenuto per 3s: Transizione ad OUT OF SERVICE e
            stop robot
            ServiceWorking = false
        }
    }
}
```

```

        OutOfServicePendingStart = -1L
        println("cargoservice | Sonar D>DFREE ($Dist >$DFree) sustained
for 3s -> OUT OF SERVICE!")
        forward("stop_robot", "stop(none)", "cargorobot")
    }
}
} else {
    // ripristino del servizio
    OutOfServicePendingStart = -1L

    if (!ServiceWorking) {
        // ripresa delle attività del robot
        ServiceWorking = true
        println("cargoservice | Sonar D<=DFREE ($Dist <=$DFree) -> SERVICE
WORKING again!")
        forward("resume_robot", "resume(none)", "cargorobot")
    }
}
}
...

```

4.6. Integrazione del LED reale

Anche il LED costituisce un dispositivo fisico e pone il medesimo problema di integrazione affrontato per il sonar.

Risulta quindi anche qui possibile evitare l'introduzione di un ulteriore componente intermedio e consentire al `cargoservice` di pubblicare direttamente i comandi sul topic MQTT dedicato al LED.

L'ESP32 si sottoscrive a tale topic e interpreta i messaggi ricevuti traducendoli nelle corrispondenti operazioni hardware (blink e spegnimento del LED).

(I riferimenti al codice sono i medesimi del sonar)

4.7. Gestione dello stato Out of Service

Nello Sprint 1 il `cargoservice` aggiornava la variabile `ServiceWorking` sulla base degli eventi del sonarmock, ma non interrompeva un movimento del robot già in corso, come invece definito successivamente dalla committente.

Nello Sprint 2 il comportamento viene completato nel seguente modo:

- se il robot è in movimento e il sistema entra in stato **Out of Service** secondo le modalità già implementate, il piano deve essere interrotto;
- quando il sonar torna a misurare `D <= D_FREE`, il sistema ritorna nello stato **Service working**;
- se un movimento era stato interrotto, deve essere ripreso.

Quando il sistema entra nello stato **Out of Service** durante una movimentazione, il `cargoservice` invia il messaggio `stop_robot` al `cargorobot`. Quest'ultimo imposta il flag `Stopped` ed emette l'evento globale `alarm(stop)`, utilizzato da `robotsmart26` per interrompere il piano in esecuzione.

```

State handle_stop {
    println("cargorobot | RICEVUTO STOP! Emetto alarm a robotsmart...") color
    red
    [# Stopped = true #]
    emit alarm : alarm(stop)
}
Goto waiting_reply if [# MoveInProgress #] else waiting_resume

```

Il cargorobot mantiene inoltre le informazioni relative al movimento in corso (TargetX, TargetY e StepTime), così da poterlo riprendere successivamente.

```

[#
    var TargetX = ""
    var TargetY = ""
    var StepTime = ""
    var Stopped = false
    var MoveInProgress = false
#]

```

Quando il sonar rileva nuovamente una condizione di Service Working, il cargoservice invia il messaggio resume_robot. Se era presente un movimento sospeso, il cargorobot reinvia la richiesta a robotsmart26, consentendo la prosecuzione della procedura.

```

State handle_resume {
    println("cargorobot | RICEVUTO RESUME! Riprendo le operazioni...") color
    green
    [# Stopped = false #]
    if [# MoveInProgress #] {
        println("cargorobot | Riprendo il movimento interrotto verso
        ($TargetX, $TargetY)...") color cyan
        request robotsmart -m moverobot : moverobot($TargetX, $TargetY,
        $StepTime)
    }
}
Goto waiting_reply if [# MoveInProgress #] else waiting

```

In questo modo il sistema può sospendere e successivamente riprendere una procedura di movimentazione senza perdere le informazioni necessarie al completamento del task.

Il link al codice di cargorobot è il seguente [link](#)^o.

4.8. Architettura finale dello Sprint 2

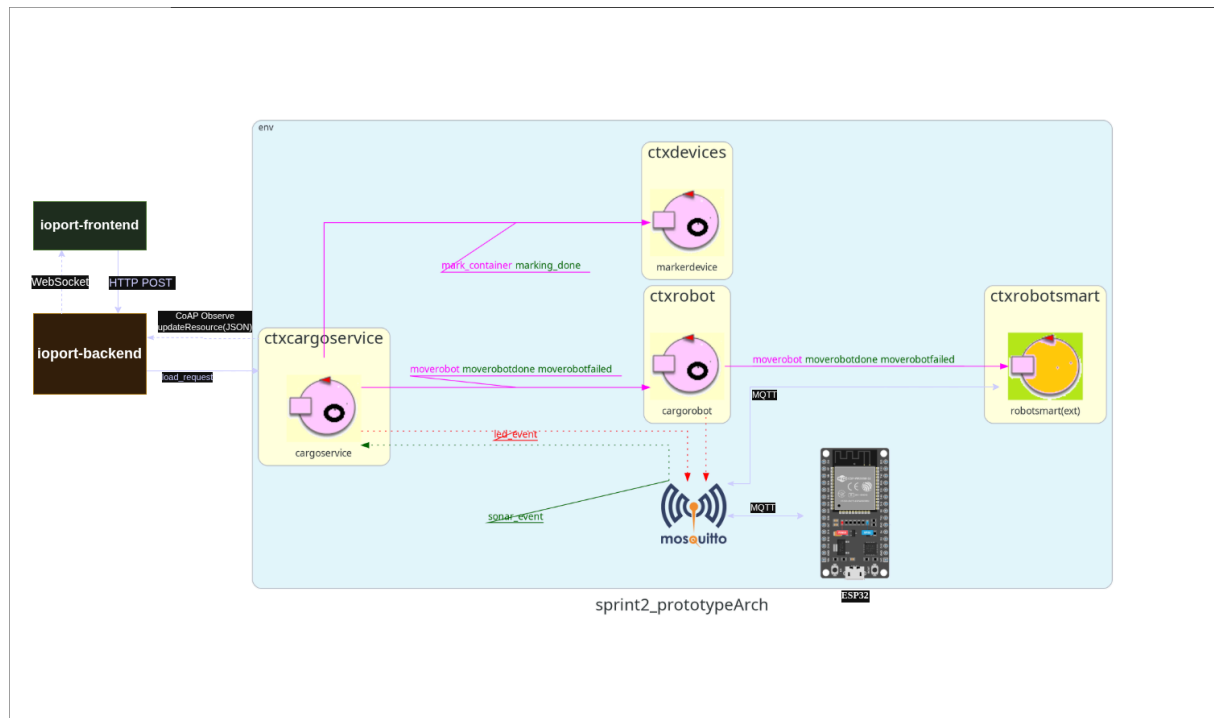


Figure 2: Architettura finale definita nello Sprint 2.

5. Test plans

5.1. Test Demo in tempo reale

Si prevede di mostrare al committente il funzionamento della piattaforma fisica, utilizzando un oggetto di prova davanti al sonar per simulare la presenza del container.

Gli scenari di test da mostrare sono i seguenti:

1. Monitoraggio dello stato

A sistema fermo viene mostrata la rilevazione dello stato corrente del sistema, verificando il passaggio tra **System Working** e **Out Of Service** e la corretta indicazione sul display. Viene quindi mostrato il rilevamento dell'IOPort come libera o occupata posizionando o rimuovendo l'oggetto davanti al sonar entro $DFREE/2$.

2. Richiesta di caricamento non disponibile

Viene effettuata una richiesta di caricamento mentre l'IOPort è occupata oppure il sistema è in stato **Out Of Service**, verificando la risposta di attesa *retrylater*.

3. Accettazione della richiesta di caricamento

Viene effettuata una richiesta di caricamento in condizioni operative, con IOPort libera, sistema funzionante e almeno uno slot disponibile. Viene verificata la risposta positiva della richiesta e l'assegnazione dello slot al container mediante lo stato *riservato*.

4. Mancato deposito del container entro il tempo previsto

Dopo una richiesta accettata, il container non viene posizionato nell'IOPort entro i 30 secondi disponibili. Viene verificato che lo slot assegnato (*riservato*) torni nuovamente disponibile (*libero*).

5. Gestione dell'interruzione durante il movimento

Viene simulata una condizione di **Out Of Service** durante il movimento del robot, verificando l'arresto e la successiva ripresa dell'operazione al ripristino del servizio.

6. Richiesta di caricamento rifiutata

Viene simulata una richiesta quando tutti gli slot disponibili sono occupati, verificando la visualizzazione del messaggio *refused* sul display.

5.2. Test Automatizzati

Il sistema è stato validato mediante una suite di test automatici sviluppata con PyTest. Può essere avviato tramite la direttiva:

```
pytest -v sprint2/tests/test_system.py
```

Gli scenari principali verificati sono i seguenti:

1. Con sistema libero (**disengaged**) viene inviata una richiesta di carico. La richiesta viene accettata e il sistema passa allo stato **engaged**.
2. Viene simulata la presenza di un container nell'IOPort e viene inviata una richiesta di carico. Il sistema risponde con **retrylater**.
3. Durante un'operazione di carico già in corso viene inviata una nuova richiesta. La richiesta viene rifiutata temporaneamente con risposta **retrylater**.
4. Dopo l'accettazione della richiesta, il container viene depositato nell'IOPort entro 30 secondi. Il caricamento prosegue correttamente senza generare un timeout.
5. Viene completato l'intero ciclo di movimentazione del robot. Il sistema ritorna allo stato **disengaged**.
6. Dopo il completamento di un carico viene inviata una nuova richiesta. Il sistema accetta correttamente la nuova operazione.
7. Viene simulata la presenza del container per meno di 3 secondi. La rilevazione viene ignorata e lo stato del sistema non cambia.
8. Dopo l'accettazione della richiesta, il container non viene depositato nell'IOPort entro 30 secondi. L'operazione viene annullata e lo slot torna disponibile.
9. Viene simulata una distanza del sonar superiore a `D <= D_FREE` per almeno 3 secondi. Il sistema riconosce la condizione di **Out Of Service**.
10. Viene inviata una richiesta mentre il sistema è nello stato **Out Of Service**. Il sistema risponde con **retrylater**.

11. Viene simulata una condizione di guasto per meno di 3 secondi. L'anomalia viene ignorata e il sistema rimane nello stato **Service Working**.
12. Dopo una condizione di **Out Of Service**, il sonar torna operativo. Il sistema ritorna allo stato **Service Working**.
13. Vengono occupati progressivamente tutti gli slot disponibili e viene inviata un'ulteriore richiesta. Il sistema rifiuta la richiesta poiché l'hold risulta pieno.

Il link al codice dei test automatici è il seguente [link](#)^o.

6. Deployment

Al fine di semplificare la manutenzione, la distribuzione e garantire modularità e isolamento dei componenti, l'architettura del sistema è stata progettata seguendo un approccio a microservizi. Si è scelto di distribuirli come container **Docker**, mentre l'intero processo di avvio ed esecuzione è orchestrato tramite **Docker Compose**.

Il sistema prevede l'esecuzione di alcuni servizi infrastrutturali e di integrazione con componenti esterni (quali **mosquitto**, il broker MQTT, e **wenv**, **RobotSmart26** e **RobotOutGui25** forniti per il controllo di base del robot e per la visualizzazione).

Si prevede poi l'avvio dei seguenti componenti (descritti nelle fasi precedenti del documento) containerizzati: **cargoservice**, **cargorobot**, **devices** e **ioport-backend**

Per semplificare l'avvio dell'intero sistema è stato predisposto uno script Bash (`start.sh`) che automatizza le operazioni di compilazione, costruzione delle immagini Docker e avvio dei container.

Lo script esegue automaticamente le seguenti operazioni (eseguibili anche manualmente):

1. Accede alle directory dei microservizi ed esegue il comando

```
./gradlew distTar
```

per compilare il progetto e generare l'archivio contenente la distribuzione dell'applicazione.

2. Crea la rete Docker interna se non esiste.
3. Costruisce le immagini utilizzando gli archivi generati nella fase precedente e avvia tutti i container in modalità detached.

```
docker compose up --build -d
```

Al termine dell'avvio, le interfacce grafiche del sistema sono accessibili dal browser ai seguenti indirizzi:

- **WebGui dell'IOPort** disponibile sulla porta locale 8086
- **Ambiente grafico del robot** disponibile sulla porta locale 8090

7. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340